MDP Project

Review 2 Package

Rough draft • Pseudocode • Software justification • Execution

Built against Review 1 feedback: be relevant to the research literature, show execution and a rough draft, add pseudocode, and justify every tool. All numbers come from the committed code on the real 204-station dataset; the framing is deliberately honest and narrowed where the evidence does not support stronger claims.

Part 1 — Rough Draft (Research Paper)

Rough Draft -- Research Paper

> ***Status:*** *rough draft for Review 2. Every number is produced by the committed code on the real dataset (see* `audit/reports/`*). Framing is deliberately honest and narrowed -- see §VIII Limitations and the claim-to-evidence table. Citations are real, foundational works (see* `audit/manuscript/BIBLIOGRAPHY.md`*); DOIs are to be confirmed against publisher records before any submission.*

Title

What Can and Cannot Be Learned from Pooled Regulatory Air-Quality Data: A Leakage-Safe Machine-Learning Case Study over 204 Indian Monitoring Stations

Alternative title: Exploratory Machine Learning on Urban Air-Quality Monitoring Data -- Predictive Baselines, Associational Structure, and Model-Based Scenarios.

> *We moved away from the original title *"Causal Machine Learning for Multi-Pollutant Source Apportionment and Control Optimization"* because our own evaluation could not support the words causal, source apportionment, or control optimization (§VIII). Narrowing the claim rather than overstating it is itself the contribution.*

Keywords: air quality; machine learning; data leakage; causal discovery; DirectLiNGAM; XGBoost; multi-objective optimization; reproducibility.

Abstract

We assemble one year (2025) of hourly observations from 204 Indian air-quality monitoring stations (OpenAQ) merged with co-located modelled meteorology (Open-Meteo) -- about 1.18 million station-hours -- and ask what a machine-learning pipeline can and cannot defensibly conclude from it. Under **leakage-safe** evaluation, a gradient-boosted model estimates contemporaneous ozone with held-out R2 ~ 0.25 (blocked-temporal) to 0.30 (unseen stations), well below the 0.41-0.43 from a leaky random split; pollutants add only ~ +0.12 R2 *beyond weather*, and a one-hour-ahead forecast beats a persistence baseline by only +0.05. Exploratory causal-structure discovery (DirectLiNGAM) recovers a single bootstrap-stable association consistent with local NO-O3 titration, but the wider graph is confounded by omitted meteorology and is not causally identified. K-means groups co-varying species only weakly (silhouette 0.26-0.39) and is not source apportionment. A linear response model admits a closed-form scenario frontier; a published metaheuristic "optimization" returned only dominated points. We conclude that the engineering is sound and reproducible, but that causal, apportionment and policy claims are unsupported by these data, and we specify the additional evidence each would require.

I. Introduction

Urban air quality in India is a major public-health problem, and open monitoring data invite machine-learning analysis. A tempting narrative runs *prediction -> causal structure -> policy optimization*. Many applied papers present each link as established. We instead **test** that narrative critically on a large but imperfect regulatory dataset and report, with uncertainty, where it holds and where it breaks.

Contributions. (1) A reproducible PySpark/Dataproc pipeline over 204 stations. (2) A leakage-safe re-evaluation showing how random splitting inflates apparent skill. (3) One stable, chemically-plausible association (NO-O3) reported as a hypothesis, not an identified effect. (4) A demonstration that a published metaheuristic "optimizer" was dominated by a closed-form solution. (5) An explicit account of the evidence each stronger claim would need.

II. Related Work (positioning)

Our methods are standard; our contribution is their *honest application and evaluation*. We position against five strands:

- **Tabular prediction / boosting.** XGBoost [Chen & Guestrin, 2016] is the canonical tabular regressor; we use it but subject it to leakage-safe evaluation rather than reporting a single random-split R2.
- **Data leakage.** [Kaufman et al., 2012] formalizes leakage; air-quality ML frequently random-splits autocorrelated time series. Our blocked-temporal and leave-stations-out protocol is the direct remedy.
- **Causal discovery.** DirectLiNGAM [Shimizu et al., 2006; 2011] and Pearl's identification framework [Pearl, 2009] let us *orient* edges; we add bootstrap stability and a meteorology-confounding test, which most applied uses omit.
- **Source apportionment.** Receptor models, esp. PMF [Paatero & Tapper, 1994], require chemical speciation and per-measurement uncertainties. Our data have neither, so we explicitly decline the apportionment claim.
- **Meteorological normalization.** [Grange & Carslaw, 2019] show weather must be removed before attributing air-quality changes -- motivating our weather-vs- pollutant decomposition and our refusal to read the raw graph as mechanism.

How our paper "matches" the literature. It uses the same methods as published causal-ML air-quality studies, but reports the diagnostics those studies usually skip (leakage-safe splits, bootstrap edge stability, confounding tests, an exact optimality check). The comparison table in `audit/manuscript/ORIGINAL_VS_CORRECTED.md` maps each original claim to its corrected, evidence-backed version.

III. Data

Hourly CO, NO, NO2, O3, PM10, PM2.5 and SO2 for 204 Indian stations with 2025 data (OpenAQ v3), each merged with hourly Open-Meteo reanalysis (temperature, relative humidity, wind speed, surface pressure) at the nearest ~0.1deg grid cell. Scale: **1,180,551 station-hours**, ~ 66% of a full 204 x 8,760 grid; 37 stations < 50% within-year coverage. Caveats: units are not preserved and thus unverified (CO is a uniform mg/m3 regime, ~ x1000 from the µg/m3 species); 35 non-physical outliers and saturation caps exist; weather is modelled; the network is urban and Delhi-NCR-heavy (not nationally representative); VOCs are structurally absent.

IV. Methodology

Full algorithmic detail is in `audit/review2/PSEUDOCODE.md`; the tool rationale is in `SOFTWARE_STACK.md`. Four analyses:

1. **Predictive baseline** -- XGBoost estimates same-hour ozone from pollutants + weather. Evaluated with leaky random (contrast only), blocked-temporal, leave-stations-out, and strict splits; preprocessing fit on train only; hyper-parameters fixed a priori; baselines = mean, climatology, persistence; uncertainty via station-block bootstrap.
2. **Structure discovery** -- DirectLiNGAM on standardized pollutants with a prior forbidding secondary->primary edges; bootstrap stability + meteorology-inclusion sensitivity.
3. **Species grouping** -- K-means on inter-species correlation profiles; silhouette model-selection + bootstrap co-assignment.

4. **Scenario analysis** -- the linear model's exact Pareto frontier derived in closed form and compared to the published metaheuristic result.

V. Experimental Setup & Execution

- **Pipeline execution.** The full three-phase pipeline runs end-to-end on Google Cloud Dataproc (custom Docker runtime) and in Cloud Shell; `main.py --phases all` completes with every phase reporting `all_ok` on the real 204-station data.
- **Audit execution.** Eight self-contained scripts (`audit/*.py`) regenerate every number offline with `SEED=42` (Track A of `RUNBOOK.md`); `pytest` guards pass.
- **Compute.** Prediction and causal-stability scripts run in ~ 2 min each on a single machine; bootstrap counts are CLI flags.

VI. Results

A. Prediction. Leakage inflates skill: random split R2 ~ 0.41, but honest evaluation gives R2 ~ 0.25 (blocked-temporal; 90% CI 0.22-0.28) and 0.30 (unseen stations; CI 0.19-0.40), down to 0.22 strict. Decomposition (Phase C2): weather alone R2 = 0.14, pollutants add **+0.12** (to 0.26), time harmonics -> 0.34; the 0.41 is only recovered by a train-only per-station mean, which is **site memorisation**, not generalisation. Persistence reaches R2 ~ 0.79-0.81 for gap-filling; a true t+1h forecast beats persistence by only **+0.05** (0.81 -> 0.87).

B. Structure. One edge is bootstrap-stable and survives meteorology: **NO -> O3, negative** (frequency 1.0, -0.138 ± 0.007; -0.108 with weather) -- local titration. PM2.5 edges collapse once weather enters (temp -> PM2.5, humidity -> O3 dominate): the pollutant-only graph was **confounded**. SO2 edges are unstable (freq <= 0.5).

C. Grouping. Silhouette weak (0.26 raw, 0.39 imputed). Core groups {NO, NO2, PM10, PM2.5} and {O3, SO2} are stable; CO's instability was an **outlier artefact** -- after flagging 35 values, CO-NO rises 0.14 -> 0.42 and CO joins the combustion group at co-assignment 1.0. Still a weak convenience partition, now physically coherent.

D. Scenarios. The frontier is a short near-linear trade-off driven almost entirely by NO. The published metaheuristic's **19 "optimal" policies are all dominated** by the exact frontier (mean PM2.5 gap 0.046, max 0.298 standardized).

VII. Discussion

The honest contribution is modest and methodological: a correctly-evaluated predictive baseline that beats spatial baselines but not temporal persistence; one stable, plausible association reported as a hypothesis; a demonstration that a published "optimization" was dominated by a closed-form solution; and a cautionary account of how leakage, omitted confounders, unharmonised units and pooling inflate apparent findings. SHAP-DirectLiNGAM agreement on the NO sign is internal consistency, **not** independent validation.

VIII. Limitations

Units unharmonised (CO); weather modelled not observed; urban non-representative network; ~ 66% completeness with mixed missingness; VOC absent; a single pooled *linear* contemporaneous model cannot represent regime-dependent (VOC- vs NOx-limited) ozone chemistry; "interventions" act on standardized ambient concentrations, not emissions. **None of the analyses identify causal effects.**

What each stronger claim would require: *causal* -> an identification strategy (natural experiment / policy discontinuity) and measured confounders; *source apportionment* -> chemical speciation + per-measurement uncertainties + VOCs and a receptor model (PMF); *control optimization* -> a validated emissions->concentration model, not a fit to ambient levels.

IX. Conclusion & Future Work

The pipeline runs, is reproducible, and the dataset is real; but its original headline claims do not survive scrutiny. Reframed as an exploratory study with leakage-safe evaluation, explicit uncertainty, and honest limitations, it is a useful cautionary contribution. Future work: verify units against OpenAQ sensor metadata; obtain speciated data for apportionment; seek a genuine intervention for causal identification; and extend the predictive model to multi-step forecasting.

References

See `audit/manuscript/BIBLIOGRAPHY.md` (Shimizu et al. 2006/2011; Pearl 2009; Chen & Guestrin 2016; Lundberg & Lee 2017; Deb et al. 2002; Akiba et al. 2019; Zaharia et al. 2016; Kaufman et al. 2012; Seinfeld & Pandis 2016; Paatero & Tapper 1994; Grange & Carslaw 2019; OpenAQ; Open-Meteo). DOIs/pages to be confirmed before submission.

Part 2 — Pseudocode

Pseudocode -- MDP Air-Quality ML Pipeline

Algorithm-level pseudocode for every stage. Each block mirrors the actual code (`data_collection.py`, `src/phases/`, `audit/`) but is written to be read and defended in a review, not executed. Notation: `D` = dataset (rows = station-hours, columns = pollutants + weather), `S` = set of stations, `u,sigma` = per-column train-fold mean/std.

Algorithm 0 -- Data Collection (`data_collection.py`)

```
INPUT:  country = India, year = 2025, pollutants P = {CO,NO,NO2,O3,PM10,PM25,SO2}
OUTPUT: one Parquet file per station in data/raw/telemetry/

for each station s in OpenAQ(country, has_data_in=year):          # v3 API
    sensors  <- GET /locations/{s}/sensors                        # capture units!
    meas     <- GET /measurements for each sensor, hourly, year
    weather  <- Open-Meteo reanalysis at nearest 0.1 deg grid cell of s
                for {temperature, relative_humidity, wind_speed, pressure}
    table    <- OUTER-JOIN(meas, weather) ON timestamp (hourly)
    write Parquet(table) with station_id, lat, lon, units-manifest
end
merge per-station files -> 204 files, ~1.18M station-hours
```

Design note: units are stored per sensor so that comparability can be *verified* later (the original collection discarded them -- a defect the audit fixed).

Algorithm 1 -- Phase 1: Species Grouping by K-means (`phase1_clustering.py`)

```
INPUT:  D, species columns C, k (default 3)
OUTPUT: cluster label per species + silhouette score

# features are per-species correlation PROFILES, not raw concentrations
R <- pairwise_correlation_matrix(D[C])          # |C| x |C|
X <- rows of R                                  # each species -> vector of its
                                                # correlations with all others
for k' in candidate range:                      # model-selection
    model   <- KMeans(k=k', seed=42).fit(X)
    score   <- silhouette(X, model.labels)
choose k with best silhouette
RETURN model.labels, score
```

Why correlation profiles: species that co-vary with the *same* other species are grouped, which is the closest K-means can get to "behaves like a common source".

Algorithm 2 -- Phase 2a: Predictive Baseline (`phase2_causal.py`, XGBoost)

```
INPUT:  D, target y = ozone, features F = other pollutants + weather
OUTPUT: fitted model + HONEST held-out skill

# --- leakage-safe splitting (the core correctness fix) ---
TRAIN, TEST <- blocked_temporal_split(D, train = months 1..9, test = 10..12)
          OR  leave_stations_out(D, S)          # unseen-station generalisation

mu, sigma   <- mean,std of F over TRAIN only    # fit preprocessing on train only
TRAIN, TEST <- standardize(F, mu, sigma)

model <- XGBoostRegressor(n_estimators=250, max_depth=6, lr=0.05,
                          subsample=0.8, tree_method=hist, seed=42)   # fixed a priori
model.fit(TRAIN[F], TRAIN[y])

R2 <- r2_score(TEST[y], model.predict(TEST[F]))
```

```
# baselines for honest comparison
R2_persist  <- r2(TEST[y], previous_hour_value)
R2_clim     <- r2(TEST[y], per_station_mean(TRAIN))
CI          <- station_block_bootstrap(R2)        # uncertainty
RETURN model, R2, R2_persist, R2_clim, CI
```

The *a priori* hyper-parameters and train-only preprocessing are what make the reported R2 trustworthy; tuning on TEST would re-introduce leakage.

Algorithm 3 -- Phase 2b: Causal Structure Discovery (DirectLiNGAM)

```
INPUT:  standardized pollutants Z, prior K (forbid secondary -> primary edges)
OUTPUT: adjacency matrix + per-edge bootstrap stability

model <- DirectLiNGAM(prior_knowledge = K).fit(Z)
A     <- model.adjacency_matrix                  # weighted DAG among species
persist manifest{columns, mu, sigma, A, threshold, seed, versions}

# stability + confounding check (the part that makes it defensible)
for b in 1..B:                                   # station-block bootstrap
    Z_b   <- resample stations with replacement
    A_b   <- DirectLiNGAM(K).fit(Z_b).adjacency
    tally edge frequencies and sign
re-fit WITH weather nodes {temp,RH,wind,pressure} as exogenous
RETURN edges with frequency, mean weight, and weather-sensitivity flag
```

An edge is reported only if it is bootstrap-stable **and** survives the inclusion of meteorology (otherwise it was confounded).

Algorithm 4 -- Phase 3a: Multi-Objective Policy Search (Optuna NSGA-II)

```
INPUT:  response model f (ozone, PM2.5 vs standardized concentration levers x)
OUTPUT: Pareto set of trade-off policies

study <- Optuna.create_study(directions=[minimize,minimize],
                             sampler = NSGAIISampler(seed=42))   # NOT the TPE default
for trial in 1..N:
    x       <- trial.suggest levers over supported variables only   # reject VOC etc.
    (o, pm) <- f.evaluate(x)                      # raise if x uses an unfitted var
    report (o, pm)
RETURN study.best_trials                           # non-dominated front
```

Guard (audit fix): decision variables are intersected with the *fitted* graph; asking to move a variable the model never learned raises, instead of silently returning a meaningless "optimum".

Algorithm 4' -- Exact frontier (audit check, `analytic_pareto.py`)

```
# because f is LINEAR, the true Pareto front is closed-form
front <- { x on the boundary of the feasible box minimizing lambda*o+(1-lambda)*pm
           for lambda in [0,1] }
for each saved metaheuristic policy p:
    assert exists q in front with q dominates p     # all 19 were dominated
```

Algorithm 5 -- Phase 3b: Explanation (SHAP)

```
INPUT:  fitted XGBoost model, background sample B
OUTPUT: per-feature attributions

explainer <- TreeSHAP(model)
phi       <- explainer.shap_values(B)              # exact for trees
rank features by mean(|phi|); inspect NO contribution to ozone
```

SHAP agreeing with DirectLiNGAM on the NO sign is **internal consistency** of two models on the same data -- reported as such, not as independent validation.

Part 3 — Software & Tools: What and Why

Software & Tools -- What We Use and Why It Matters

For each component: **what it is**, **why we chose it over alternatives**, and **where it is used** in the pipeline. This answers the review requirement to "explain the importance of each code and software, and why we are using it."

Infrastructure / Big-data engine

Apache Spark (PySpark)

- **What:** distributed in-memory data-processing engine with a Python API.
- **Why it matters:** the dataset is ~1.18M station-hours across 204 files; Spark lets the same code run on a laptop and scale out on a cluster without rewriting. Its DataFrame API and lazy execution plan avoid loading everything into RAM.
- **Why not pandas alone:** pandas is single-machine and would not scale to the full multi-city/multi-year target; Spark is the standard for this scale.
- **Used in:** all three pipeline phases (`src/phases/`), driven by `main.py`.

Google Cloud Dataproc

- **What:** managed Spark/Hadoop clusters on Google Cloud.
- **Why it matters:** gives a reproducible, disposable cluster so a reviewer can re-run the exact pipeline; billed per-second, so experiments are cheap and nothing persists accidentally.
- **Why not a static VM/Hadoop install:** no cluster to maintain; the custom runtime image (`Dockerfile`) pins every dependency version for reproducibility.
- **Used in:** `submit.sh`, `cloudbuild.yaml`, `Dockerfile` (deployment).

Parquet (via PyArrow)

- **What:** columnar, compressed, typed on-disk format.
- **Why it matters:** column pruning + compression make reading a few pollutants out of 204 files fast; the embedded schema prevents silent type drift.
- **Why not CSV:** CSV loses types and is far larger/slower; a real bug we hit (int-vs-float humidity) is exactly what a typed format guards against.

Machine-learning methods

K-means clustering (Spark MLlib)

- **What:** partitions items into `k` groups by minimizing within-cluster variance.
- **Why it matters:** a transparent, fast first pass at "which species behave alike"; the silhouette score gives an objective number of groups.
- **Why not DBSCAN/hierarchical here:** we cluster a handful of species on correlation profiles -- K-means with silhouette model-selection is simple, deterministic (seeded) and defensible; it is **grouping, not apportionment**.
- **Used in:** Phase 1 (`phase1_clustering.py`).

XGBoost (SparkXGBRegressor)

- **What:** gradient-boosted decision trees -- a strong tabular regressor.
- **Why it matters:** air-quality relationships are nonlinear and interacting; boosted trees capture these without manual feature engineering and are the consistent top performer on tabular benchmarks.

- **Why not linear regression / deep nets:** linear underfits the chemistry; deep nets need far more data and tuning and rarely beat boosting on tabular data.
- **Used in:** Phase 2a predictive baseline, and the SHAP explanation.

DirectLiNGAM (`lingam`)

- **What:** learns a linear, non-Gaussian acyclic causal structure (a DAG) from observational data.
- **Why it matters:** it is one of the few methods that can *orient* edges (not just correlate) under its assumptions, letting us pose directional hypotheses such as NO -> ozone.
- **Why we are cautious:** it assumes linearity, non-Gaussianity, acyclicity and **no hidden confounders** -- we explicitly test the last by adding meteorology, which is why we report only the one edge that survives.
- **Used in:** Phase 2b causal discovery.

Optuna + NSGA-II sampler

- **What:** Optuna is an optimization framework; NSGA-II is a multi-objective evolutionary algorithm that returns a Pareto front, not one answer.
- **Why it matters:** policy involves trade-offs (lower ozone vs lower PM2.5); NSGA-II surfaces the whole trade-off surface instead of a single compromise.
- **Why the sampler choice is critical:** Optuna defaults to TPE, which is single-objective; we must pass `NSGAIISampler` explicitly (a bug the audit fixed). For our *linear* model we also derived the exact front in closed form to check the metaheuristic -- and it was dominated.
- **Used in:** Phase 3a optimization + `analytic_pareto.py` cross-check.

SHAP (TreeExplainer)

- **What:** Shapley-value feature attributions; exact and fast for tree models.
- **Why it matters:** turns the black-box XGBoost into per-feature, per-prediction explanations with a sound game-theoretic basis -- needed for any "XAI" claim.
- **Why not plain feature importance:** gain/weight importances are inconsistent and global; SHAP is locally accurate and additive.
- **Used in:** Phase 3b explainability.

Data sources

OpenAQ v3 API

- **What:** open aggregator of government/reference air-quality monitors.
- **Why it matters:** real, citable regulatory measurements for India at hourly resolution -- no fabrication, and reproducible by any reviewer with a free key.
- **Caveat we surface:** per-sensor units must be captured to compare stations.

Open-Meteo reanalysis

- **What:** free historical/reanalysis weather API.
- **Why it matters:** ozone is weather-driven; we need co-located temperature, humidity, wind and pressure. Reanalysis gives complete, gap-free coverage.
- **Caveat we surface:** weather is **modelled**, not measured on-site.

Engineering & reproducibility

Tool	Why it matters
Git + feature branch	Raw data and original results preserved; all audit work isolated on `review/scientific-audit`.

Tool	Why it matters
Docker (custom image)	Pins every library version so the cluster run is byte-for-byte reproducible.
Fixed seeds (`SEED=42`**)**	Every clustering/boosting/bootstrap run is deterministic and re-checkable.
scikit-learn / SciPy / statsmodels	Standard, audited implementations of metrics, splits, bootstrap and stats used throughout the audit.
Matplotlib	Publication figures (e.g. the scenario frontier) generated from the same data.
pytest	Encodes correctness guards (e.g. "optimizer front truly dominates") as tests that must pass.

One-line summary of the stack choice: every tool is the *standard, citable, reproducible* option for its job -- Spark/Dataproc for scale, Parquet for typed storage, XGBoost for tabular prediction, DirectLiNGAM for directional hypotheses, NSGA-II for trade-offs, SHAP for explanations -- and each is paired with an honest caveat about what it can and cannot establish on this dataset.